

TECHNICAL TRAINING COURSE

Dispatching Requests to Methods

JAX-RS 2.x

- `@ApplicationPath` · `@Path` templates · Regex constraints
- HTTP method annotations · Content negotiation
- Sub-resource locators · Annotation inheritance

Java 11 LTS

Tomcat 9.0

JAX-RS 2.1

Jersey 2.x



COURSE AGENDA

5 Sections | Request Dispatch Deep Dive

01

The `ApplicationPath`

02

The `@Path` Annotation

03

HTTP Method Annotations

04

Sub-resource Locators

05

Annotation Inheritance and Overriding

The Three-Part URI — How Dispatch Begins



```
http://host:8080 /shop /api /products/P001
```

- @ApplicationPath must start with /
- Do not include a trailing /
- Runtime appends `/*` when registering the Servlet
- Two Applications must have different paths

@ApplicationPath in Action

```
// Minimal – classpath scanning discovers @Path and @Provider classes
@ApplicationPath("/api")
public class ShopApplication extends Application { }

// Versioned – two apps in one WAR, each at a different path
@ApplicationPath("/api/v1")
public class ApiV1Application extends Application {
    @Override public Set<Class<?>> getClasses() {
        return Set.of(V1ProductResource.class, V1OrderResource.class);
    }
}

@ApplicationPath("/api/v2")
public class ApiV2Application extends Application {
    @Override public Set<Class<?>> getClasses() {
        return Set.of(V2ProductResource.class, V2OrderResource.class);
    }
}

// KEY RULE: returning ANY non-empty Set disables classpath scanning!
```

@Path — Class Level and Method Level

```
@Path("/orders")                // class: base path
public class OrderResource {

    @GET                        // GET /orders
    public List<Order> list() { ... }

    @GET
    @Path("{id}")              // GET /orders/{id}
    public Order get(
        @PathParam("id") String id) { ... }

    @GET
    @Path("{id}/status")       // GET /orders/{id}/status
    public String status(
        @PathParam("id") String id) { ... }

    @POST
    @Path("{id}/lines")        // POST /orders/{id}/lines
    public Response addLine(
        @PathParam("id") String id,
        OrderLine line) { ... }
}
```

1

Full path = class @Path + method @Path

2

Slashes at boundaries are normalised automatically

3

{variable} captures any non-slash sequence

4

@PathParam converts String to target type

5

Type conversion failure → 404

Regex-Constrained Templates & Matching Priority

```
// Digits only: GET /products/42
@GET @Path("{id: \\d+}")
public Product getById(@PathParam("id") int id){ }

// SKU pattern: GET /products/WIDGET-001
@GET @Path("{sku: [A-Z]+-\\d+}")
public Product getBySku(@PathParam("sku") String s){ }

// Greedy path: GET /files/a/b/c.txt
@GET @Path("files/{path: .+}")
public Response getFile(@PathParam("path") String p){ }
```

Practical Rule:

A more literal (specific) path always beats a template.
`/orders/pending` beats `/orders/{id}` for GET `/orders/pending`

Matching Tiebreaker Rules

1

More literals win

`/orders/active` beats `/orders/{id}`

2

Fewer variables win

`/orders/{id}/s` beats
`/orders/{a}/{b}`

3

Regex beats plain

`{id: \\d+}` beats `{id}`

4

Media type specificity

`application/xml` beats `/*/*`

HTTP Method Annotations

Annotation	Safe	Idempotent	Typical Use
<code>@GET</code>	Yes	Yes	Retrieve resource / collection
<code>@POST</code>	No	No	Create resource or trigger action
<code>@PUT</code>	No	Yes	Full replacement of a resource
<code>@PATCH</code>	No	No	Partial update of a resource
<code>@DELETE</code>	No	Yes	Remove a resource
<code>@HEAD</code>	Yes	Yes	Same as GET, no body
<code>@OPTIONS</code>	Yes	Yes	Allowed methods / CORS pre-flight

Response Status Codes by Convention

Method	Status Code	Notes
@GET (found)	200 OK	Body contains the representation
@GET (not found)	404 Not Found	Resource does not exist
@POST (create)	201 Created	Location header → new resource URI
@POST (action)	200 OK	Result in body
@PUT / @PATCH	200 OK	Updated representation in body
@DELETE	204 No Content	No body expected
Wrong verb	405 Method Not Allowed	Path exists but not for that verb
Bad Content-Type	415 Unsupported Media Type	No method @Consumes that type
Bad Accept	406 Not Acceptable	No method @Produces matching type

Sub-resource Locators

A locator returns a handler — the runtime dispatches to the returned object

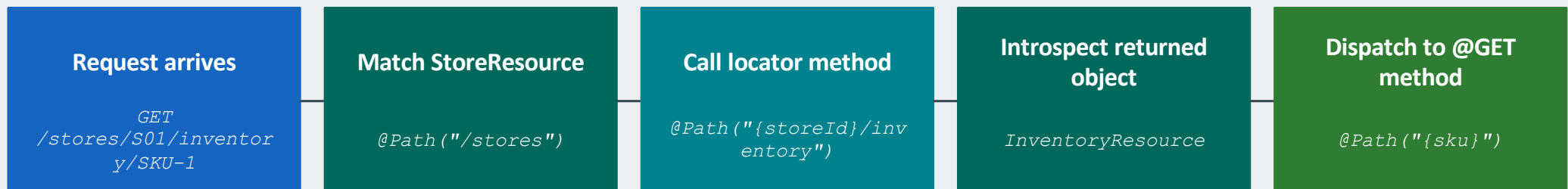
Sub-resource METHOD (@Path + verb)

```
// Has @GET — handles request
@GET
@Path("/{id}/status")
@Produces(TEXT_PLAIN)
public String getStatus(
    @PathParam("id") String id){
    return svc.find(id).getStatus();
}
```

Sub-resource LOCATOR (@Path only)

```
// NO verb — returns a handler
@Path("/{storeId}/inventory")
public InventoryResource locator(
    @PathParam("storeId") String id){
    Store s = storeSvc.find(id);
    if(s==null)
        throw new NotFoundException();
    return new InventoryResource(s);
}
```

Locator Dispatch Flow & CDI Injection



CDI injection does NOT work in instances created with 'new'. Use `ResourceContext.initResource()` to enable it.

```
// Enable CDI injection in the sub-resource
@Path("{storeId}/inventory")
public InventoryResource locator(@PathParam("storeId") String id,
                                @Context ResourceContext rc) {
    InventoryResource resource = new InventoryResource();
    rc.initResource(resource); // triggers CDI injection
    resource.setStore(storeSvc.find(id));
    return resource;
}
```

Annotation Inheritance Rules

Class-level @Path

IS inherited

Subclass with no @Path inherits from superclass

Interface @Path

IS inherited

Implementing class inherits interface annotations

Method @GET / @Path

NOT inherited

Subclass override must redeclare ALL method annotations

Partial method override

NOT supported

If ANY annotation added → ALL parent annotations ignored

@Provider on superclass

NOT inherited

Each provider class needs its own @Provider

@PathParam on interface

IS inherited

Parameter annotations propagate with the method contract

Interface Contract + Implementation — Best Practice

```
// Interface — REST contract
@Path("/products")
@Produces(APPLICATION_XML)
@Consumes(APPLICATION_XML)
public interface ProductService {

    @GET
    List<Product> listAll();

    @GET @Path("{id}")
    Product findById(
        @PathParam("id") String id);

    @POST
    Response create(Product p,
        @Context UriInfo ui);
}
```

```
// Impl — inherits @Path, @Produces
public class ProductResource
    implements ProductService {

    @Inject ProductRepository repo;

    // @GET inherited — don't redeclare
    @Override
    public List<Product> listAll() {
        return repo.findAll();
    }

    @Override
    public Product findById(String id){
        return repo.findById(id);
    }

    @Override
    public Response create(
        Product p, UriInfo ui){ ... }
}
```

Method Override — All or Nothing

CRITICAL: If a subclass method adds ANY JAX-RS annotation, ALL parent annotations are ignored. No partial inheritance.

```
// Base class
@Path("/reports")
public class ReportResource {

    @GET
    @Produces(APPLICATION_XML)
    public List<Report> list() { ... }
}

// Subclass — CORRECT override
@Path("/admin/reports")
public class AdminReportResource
    extends ReportResource {

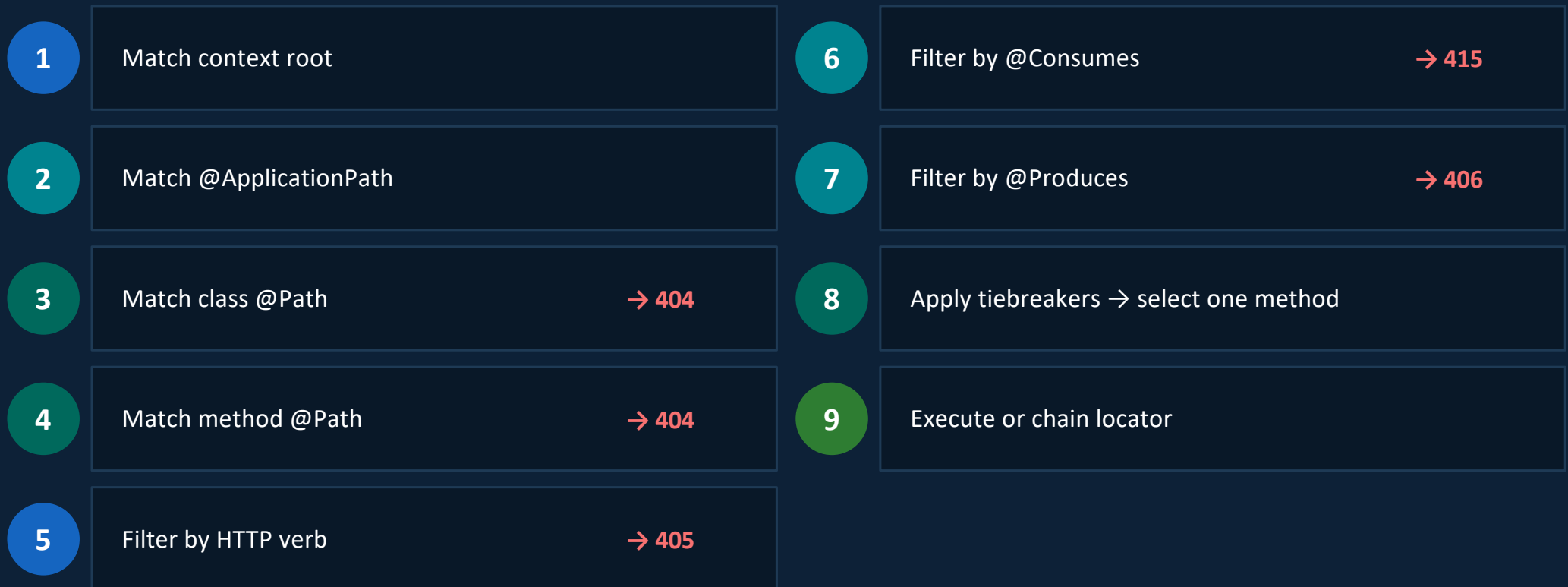
    @Override
    @GET // must redeclare
    @Produces({APPLICATION_XML,
               "text/csv"}) // add CSV
    public List<Report> list() {
        return super.list();
    }
}
```

```
// WRONG — @GET is lost!
// list() becomes a locator,
// not a resource method.

@Override
// Missing @GET ← PROBLEM
@Produces({APPLICATION_XML,
           "text/csv"})
public List<Report> list() {
    return super.list();
}

// This method is now invisible
// to JAX-RS dispatch — no verb
// annotation means it is
// treated as a sub-resource
// locator, not a @GET handler.
```

Complete Dispatch Decision Flow



Key Takeaways

01

Full URI = context root + @ApplicationPath + @Path (class + method).

02

Literals beat template variables; regex beats plain templates in matching.

03

Each HTTP verb has a corresponding annotation; safe/idempotent properties drive API design.

04

A locator has @Path but no verb — it returns an object the runtime then dispatches to.

05

Class annotations inherit; method annotations do NOT — all or nothing on subclass override.

06

Dispatch errors: 404 (no match), 405 (wrong verb), 415 (Content-Type), 406 (Accept).

Thank You

Questions & Lab Time

Lab exercises:

- Lab 1: `@ApplicationPath` — deploy two API versions
- Lab 2: `@Path` templates — literals, variables, regex
- Lab 3: HTTP methods — CRUD + content negotiation
- Lab 4: Sub-resource locators — hierarchy design
- Lab 5: Annotation inheritance — interface contracts

